

Part 1: The Head Node

Role: Gateway, Login Node, NFS Server.

OS: Ubuntu Desktop 24.04 LTS.

1. OS Installation & User Setup

- **Install:** Standard Ubuntu Desktop 24.04.
- **Create Default User:** Name it `student`. (Ubuntu automatically assigns UID `1000` to the first user. We will use this exact user across all machines).

2. Network Configuration

Using the terminal, set up the Ethernet port (replace `eth0` with your actual ethernet interface name, e.g., `enp0s31f6`):

```
sudo nmcli connection add type ethernet ifname eth0 con-name cluster-lan ipv4.method manual ipv4.address 10.0.0.1/24  
sudo nmcli connection up cluster-lan
```

3. Enable Internet Sharing (NAT)

Share the head node's Wi-Fi (`wlan0`) with the compute nodes:

```
sudo sysctl -w net.ipv4.ip_forward=1  
sudo iptables -t nat -A POSTROUTING -o wlan0 -j MASQUERADE  
sudo iptables -A FORWARD -i wlan0 -o eth0 -m state --state RELATED,ESTABLISHED -j ACCEPT  
sudo iptables -A FORWARD -i eth0 -o wlan0 -j ACCEPT  
sudo apt install iptables-persistent # Select "Yes" to save rules when prompted
```

4. Core Software Stack

Install the compilers, MPI, and ATLAS libraries:

```
sudo apt update  
sudo apt install build-essential gcc g++ openmpi-bin openmpi-common libopenmpi-dev libatlas-base
```

Part 2: The Compute Nodes

Role: Number Crunching.

OS: Ubuntu Desktop 24.04 LTS.

1. Installation & User Setup

- **Install:** Standard Ubuntu Desktop 24.04.
- **User:** Create the exact same `student` user during installation to guarantee UID `1000`.

2. Network Configuration

Set static IPs for each compute node.

Node 1:

```
sudo nmcli connection add type ethernet ifname eth0 con-name cluster-lan ipv4.method manual ipv4.address 10.0.0.2/24  
sudo nmcli connection up cluster-lan
```

(Repeat for Node 2 using `10.0.0.3/24` and Node 3 using `10.0.0.4/24`)

3. Core Software Stack

Install the exact same stack as the head node:

```
sudo apt update  
sudo apt install build-essential gcc g++ openmpi-bin openmpi-common libopenmpi-dev libatlas-base
```

Part 3: The Cluster Fabric (Linking Them)

1. Internal DNS (`/etc/hosts`)

Edit `/etc/hosts` on **ALL 4 NODES** and add these lines:

```
10.0.0.1    headnode  
10.0.0.2    node01  
10.0.0.3    node02  
10.0.0.4    node03
```

2. Passwordless SSH

On the **Head Node** (as user `student`), generate an SSH key and share it:

```
ssh-keygen -t rsa -N ""
ssh-copy-id node01
ssh-copy-id node02
ssh-copy-id node03
```

3. The Shared `/mirror` Directory (NFS)

This creates a dedicated workspace `/mirror` shared across the cluster.

- **Head Node (Server):**

```
sudo apt install nfs-kernel-server
sudo mkdir /mirror
sudo chown student:student /mirror
echo "/mirror 10.0.0.0/24(rw,sync,no_root_squash,no_subtree_check)" | sudo tee -a /etc/exports
sudo exportfs -a
sudo systemctl restart nfs-kernel-server
```

- **Compute Nodes (Clients):**

```
sudo apt install nfs-common
sudo mkdir /mirror
echo "headnode:/mirror /mirror nfs defaults 0 0" | sudo tee -a /etc/fstab
sudo mount -a
```

(Test this by creating a file in `/mirror` on the head node and checking if it appears in `/mirror` on the compute nodes).

Part 4: Testing and Verification

All your cluster work should now happen inside `/mirror` so that all nodes can access the compiled programs and hostfiles.

1. Create the MPI Hostfile

On the Head Node, navigate to the mirror and create a file named `hostfile` :

```
cd /mirror
nano hostfile
```

Add your nodes and their core counts (assuming 8 cores each):

```
node01 slots=8
node02 slots=8
node03 slots=8
```

2. Run `hostname` on All Nodes

To verify that MPI can reach all nodes, run:

```
mpirun --hostfile hostfile --map-by node -np 3 hostname
```

Success: You should see `node01` , `node02` , and `node03` printed.

3. Verify with a C Program

Still in `/mirror` , create `mpi_test.c` :

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    MPI_Init(NULL, NULL);
    int world_size, world_rank, name_len;
    char processor_name[MPI_MAX_PROCESSOR_NAME];

    MPI_Comm_size(MPI_COMM_WORLD, &world_size);
    MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);
    MPI_Get_processor_name(processor_name, &name_len);

    printf("Hello from processor %s, rank %d out of %d processors\n", processor_name, world_rank, world_size);
    MPI_Finalize();
}
```

Compile and run it:

```
mpicc mpi_test.c -o mpi_test
mpirun --hostfile hostfile -np 12 ./mpi_test
```

Part 5: Benchmarking with HPL (Linpack)

1. Download and Compile HPL with ATLAS

Run these commands on the Head Node inside your shared directory:

```
cd /mirror
wget https://www.netlib.org/benchmark/hpl/hpl-2.3.tar.gz
tar xf hpl-2.3.tar.gz
cd hpl-2.3
cp setup/Make.Linux_PII_FBLAS Make.Ubuntu
```

Edit `Make.Ubuntu` to link against GCC, OpenMPI, and ATLAS:

```
ARCH          = Ubuntu
CC            = mpicc
CCFLAGS       = $(HPL_DEFS) -O3 -fomit-frame-pointer -funroll-loops -W -Wall
LINKER        = mpicc
LINKFLAGS     = $(CCFLAGS)

# Point to ATLAS libraries
LAdir         = /usr/lib/x86_64-linux-gnu
LAlib         = -L$(LAdir) -lcblas -latlas
```

Compile:

```
make arch=Ubuntu
```

The executable `xhpl` will be generated in `bin/Ubuntu/`.

2. Tuning `HPL.dat`

Navigate to `bin/Ubuntu/`. The `HPL.dat` file dictates the test parameters.

- **N (Problem Size):** Uses about 80% of total available RAM. Formula: $N \approx \sqrt{\frac{\text{Total Memory in Bytes} \times 0.8}{8}}$ (e.g., for 1 node with 16GB RAM, $N \approx 40000$).
- **NB (Block Size):** Set to 192 or 256.
- **P and Q (Grid Size):** $P * Q$ must equal your total MPI processes. Keep P slightly smaller than or equal to Q .

3. Running the Benchmark

Test 1: Single Node

- Edit `HPL.dat` : Set $N=40000$, $P=2$, $Q=4$ (assuming 8 processes).
- Run: `mpirun --hostfile /mirror/hostfile -np 8 ./xhpl`
- **Record:** Look for the **Gflops** column in the output.

Test 2: Two Nodes

- Edit `HPL.dat` : Set $N=56000$, $P=4$, $Q=4$ (assuming 16 processes).
- Run: `mpirun --hostfile /mirror/hostfile -np 16 ./xhpl`
- **Record:** The Gflops value.

Test 3: Three Nodes

- Edit `HPL.dat` : Set $N=69000$, $P=4$, $Q=6$ (assuming 24 processes).
- Run: `mpirun --hostfile /mirror/hostfile -np 24 ./xhpl`
- **Record:** The Gflops value.

4. Plotting Results

Calculate scaling:

- **Speedup:** Performance on N nodes / Performance on 1 node.
- **Efficiency:** (Speedup / Number of nodes) * 100.

Part 6: Optional Add-on (Slurm Workload Manager)

If you implement Slurm later, jobs will also run out of `/mirror`.

1. **Munge (All Nodes):** `sudo apt install munge`. Sync `/etc/munge/munge.key` from head to compute nodes.

2. **Controller (Head Node):** `sudo apt install slurmctld slurm-client` . Configure `/etc/slurm/slurm.conf` .
3. **Client (Compute Nodes):** `sudo apt install slurmd slurm-client` . Copy `slurm.conf` from the head node.
4. **Start Services:** Enable `slurmctld` (head) and `slurmd` (compute).